



FASE TRANSFORM — Dokumentasi

Pembelajaran ETL Pipeline

File Source Code: [src/transform.py](#) **Terakhir Diperbarui:** Juni 2026 **Level:** Beginner to Intermediate **Estimasi Waktu Belajar:** 2-3 Jam



Daftar Isi

1. Pendahuluan (Apa Itu Transform?)
2. Arsitektur & Pipeline Transform
3. Konsep Penting: `df.copy()` — Mengapa Selalu Membuat Salinan?
4. Fungsi `standardization_text()` — Penjelasan Baris Per Baris
5. Fungsi `fix_data_type()` — Penjelasan Baris Per Baris
6. Fungsi `handling_missing_values()` — Penjelasan Baris Per Baris
7. Fungsi `handling_duplicates()` — Penjelasan Baris Per Baris
8. Fungsi `transform_all()` — Orchestrator Pipeline
9. Blok `if __name__ == "__main__"`
10. Konsep Python Yang Digunakan
11. Data Quality Metrics
12. Ringkasan & Poin-Poin Penting

1. Pendahuluan (Apa Itu Transform?)

1.1 Definisi Transform dalam Konteks ETL

Dalam pipeline **ETL (Extract, Transform, Load)**, fase **Transform** adalah tahap kedua di mana data mentah (raw data) yang sudah diekstrak dari sumber data akan **dibersihkan, diubah, distandarisasi, dan dipersiapkan** sebelum dimuat (load) ke tujuan akhir.

Bayangkan analogi sederhana:

| Analogi | ETL | |-----|-----| | Pabrik menerima bahan mentah | **Extract** — mengambil data dari sumber | | Bahan mentah dibersihkan, dipotong, dibentuk | **Transform** — membersihkan & memperbaiki data | | Produk jadi dikemas dan dikirim | **Load** — menyimpan data ke tujuan akhir |

Transform adalah **jantung dari ETL pipeline**. Tanpa transformasi yang baik, data yang disimpan akan kotor, tidak konsisten, dan tidak bisa diandalkan untuk analisis.

1.2 Mengapa Transform Adalah Fase Paling Kompleks?

Transform dianggap fase **paling kompleks** dalam ETL karena beberapa alasan:

1. **Variasi Masalah Data:** Setiap dataset memiliki masalah unik — missing values, duplikat, format tidak konsisten, tipe data salah, whitespace tersembunyi, dan lain-lain.
2. **Keputusan Bisnis:** Banyak keputusan transformasi bergantung pada context bisnis. Misalnya: apakah missing value harus dihapus atau diisi? Jawabannya tergantung kebutuhan analisis.
3. **Urutan Operasi Penting:** Urutan transformasi mempengaruhi hasil akhir. Salah urutan bisa menyebabkan data loss atau error.
4. **Volume Data:** Transformasi harus efisien saat memproses jutaan baris data.

1.3 Jenis-Jenis Transformasi Data

| Jenis Transformasi | Deskripsi | Contoh di Kode Kita | |-----|-----|-----| | 🧹 **Data Cleaning** | Membersihkan data kotor | `handling_missing_values()`, `handling_duplicates()` | | 🗑️ **Standardization** | Menyeragamkan format data | `standardization_text()` — menghapus whitespace | | 🔄 **Type Conversion** | Mengubah tipe data | `fix_data_type()` — konversi ke datetime dan Int64 | | 📊 **Enrichment** | Menambah data baru dari sumber lain | *(Tidak ada di kode ini, tapi contoh: menambah kolom negara berdasarkan kode pos)* | | 📈 **Aggregation** | Merangkum data | *(Tidak ada di kode ini, tapi contoh: total penjualan per bulan)* |

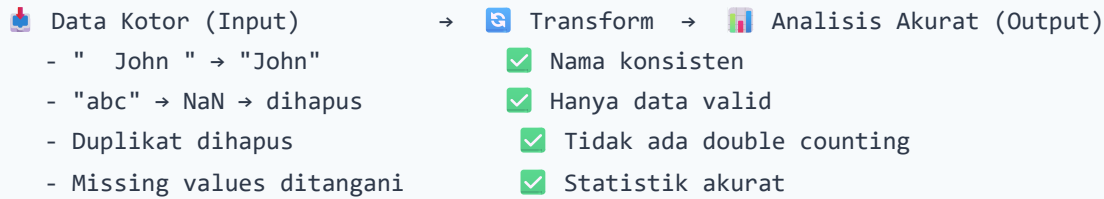
1.4 Prinsip "Garbage In, Garbage Out" (GIGO)

"Garbage In, Garbage Out" — Jika kita memasukkan data sampah, maka output-nya juga sampah.

Prinsip ini adalah fondasi utama mengapa fase Transform sangat krusial:

🗑️ Data Kotor (Input)	→	📊 Analisis Salah (Output)
- " John " (ada spasi)		- "John" dan " John " dihitung sebagai 2 orang berbeda
- CustomerID = "abc"		- Perhitungan statistik gagal
- Baris duplikat		- Total penjualan membengkak (double counting)
- Missing values		- Rata-rata menjadi bias

Dengan Transform yang benar:



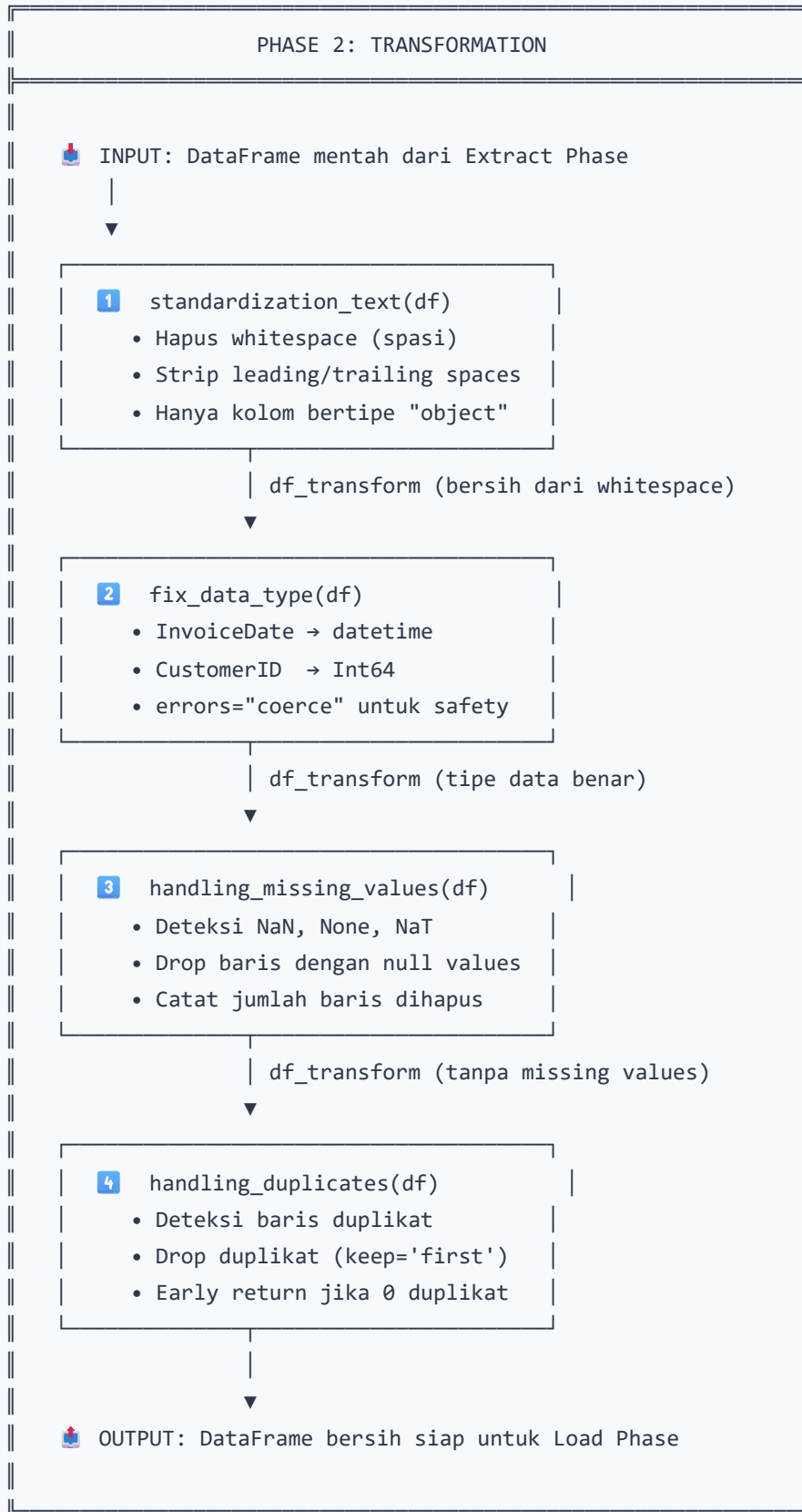
1.5 Perbedaan Transform vs Extract

| Aspek | Extract (Fase 1) | Transform (Fase 2) | |-----|-----|-----| | **Tujuan** | Mengambil data dari sumber | Membersihkan dan memperbaiki data | | **Input** | File CSV, database, API | DataFrame mentah dari Extract | | **Output** | DataFrame mentah (raw) | DataFrame bersih (clean) | | **Modifikasi Data?** | ✗ Tidak — hanya membaca | ✓ Ya — mengubah, membersihkan, menghapus | | **Kompleksitas** | Relatif sederhana | Paling kompleks | | **Error Handling** | File tidak ditemukan, format salah | Data kotor, tipe salah, missing values |

2. 🏗️ Arsitektur & Pipeline Transform

2.1 Diagram Alur Pipeline Transformasi

Berikut adalah diagram alur lengkap dari pipeline transformasi yang digunakan di `transform.py` :



2.2 Urutan Pipeline dan Mengapa Urutan Ini Penting

Pipeline transformasi di kode kita memiliki urutan yang **sangat disengaja**:

```
# Pipeline Transform Phase (dari transform_all)
df_transform = standardization_text(df_transform) # 1 PERTAMA
df_transform = fix_data_type(df_transform) # 2 KEDUA
df_transform = handling_missing_values(df_transform) # 3 KETIGA
df_transform = handling_duplicates(df_transform) # 4 KEEMPAT
```

Mengapa urutan ini penting? Mari kita analisis satu per satu:

1 `standardization_text` HARUS pertama

Bayangkan data seperti ini:

```
CustomerID: " 12345 " (ada spasi di depan dan belakang)
```

Jika kita langsung jalankan `fix_data_type` tanpa strip whitespace dulu:

```
pd.to_numeric(" 12345 ", errors="coerce") # Hasilnya: 12345.0 ✓ (kebetulan berhasil)
# TAPI bagaimana jika:
pd.to_numeric(" abc ", errors="coerce") # Hasilnya: NaN
# Jika sudah di-strip: "abc" → tetap NaN, tapi setidaknya kita tahu datanya memang salah
# bukan karena whitespace yang mengganggu
```

Lebih penting lagi, untuk **deteksi duplikat**:

```
"John" dan " John " → BERBEDA jika belum di-strip!
"John" dan "John" → SAMA setelah di-strip ✓
```

Jadi **strip whitespace harus dilakukan paling awal** agar proses selanjutnya bekerja dengan data yang konsisten.

2 `fix_data_type` HARUS sebelum handling missing values

Ketika kita mengonversi tipe data dengan `errors="coerce"`, data yang tidak valid akan diubah menjadi `NaN` (atau `NaT` untuk datetime). Contoh:

```
pd.to_numeric("abc", errors="coerce") # → NaN (nilai baru yang missing!)
pd.to_datetime("not-a-date", errors="coerce") # → NaT (Not a Time)
```

`NaN` dan `NaT` baru ini **harus ditangani** di langkah berikutnya (`handling_missing_values`). Jika urutan dibalik, missing values dari konversi tipe data yang gagal **tidak akan tertangani**.

3 `handling_missing_values` HARUS sebelum handling duplicates

Setelah missing values dihapus, jumlah baris berkurang. Ini berarti:

- **Proses deteksi duplikat menjadi lebih cepat** (lebih sedikit baris yang harus dibandingkan)
- **Menghindari false negatives**: baris dengan `NaN` mungkin terlihat unik padahal sebenarnya duplikat jika `NaN`-nya diisi

4 `handling_duplicates` terakhir

Ini adalah **langkah pembersihan terakhir**, memastikan tidak ada baris identik yang tersisa setelah semua normalisasi selesai.

2.3 Ringkasan Urutan (Tabel)

Urutan	Fungsi	Alasan Posisi
1	<code>standardization_text()</code>	Whitespace harus bersih agar konversi tipe dan deteksi duplikat akurat
2	<code>fix_data_type()</code>	Konversi gagal menghasilkan <code>NaN/NaT</code> baru yang perlu ditangani
3	<code>handling_missing_values()</code>	Hapus semua <code>NaN/NaT</code> (termasuk dari langkah 2)
4	<code>handling_duplicates()</code>	Deteksi duplikat paling akurat setelah data bersih dan konsisten

3. 🗝️ Konsep Penting: `df.copy()` — Mengapa Selalu Membuat Salinan?

3.1 Apa Itu Mutable vs Immutable di Python?

Di Python, objek dibagi menjadi dua kategori:

Kategori	Contoh	Perilaku
Immutable (Tidak bisa diubah)	<code>int</code> , <code>float</code> , <code>str</code> , <code>tuple</code>	Setiap perubahan membuat objek baru
Mutable (Bisa diubah)	<code>list</code> , <code>dict</code> , <code>set</code> , <code>DataFrame</code>	Perubahan langsung mengubah objek asli

DataFrame dari Pandas adalah MUTABLE — artinya jika kamu mengubah DataFrame, perubahan itu bisa mempengaruhi **semua variabel yang menunjuk ke DataFrame yang sama**.

```
# Contoh bahaya mutable object
import pandas as pd

df_original = pd.DataFrame({"Nama": ["Andi", "Budi"]})
df_copy = df_original # ⚠️ INI BUKAN SALINAN! Ini hanya referensi baru!

df_copy["Nama"][0] = "Charlie"

print(df_original)
# Output: Nama
#      Charlie ← 🧑 Data original IKUT BERUBAH!
#      Budi
```

3.2 Shallow Copy vs Deep Copy

| Jenis Copy | Penjelasan | Pandas | |-----|-----|-----| | **Reference (bukan copy)** | `df_new = df` — hanya membuat nama baru untuk objek yang sama | ❌ Berbahaya | | **Shallow Copy** | Menyalin struktur luar, tapi isi dalam masih berbagi referensi | `df.copy(deep=False)` | | **Deep Copy** | Menyalin **semua** — struktur dan isinya benar-benar independen | `df.copy()` atau `df.copy(deep=True)` |

Secara default, `df.copy()` melakukan **deep copy**. Ini adalah pilihan yang paling aman.

3.3 Mengapa `df.copy()` Penting untuk Menghindari `SettingWithCopyWarning`

Pandas memiliki peringatan terkenal bernama `SettingWithCopyWarning`. Peringatan ini muncul ketika kamu mencoba mengubah data pada sesuatu yang **mungkin** merupakan view (bukan salinan) dari DataFrame asli.

```
# ⚠️ Kode yang MEMICU SettingWithCopyWarning:
df_subset = df[df["Quantity"] > 0] # Ini MUNGKIN view, MUNGKIN copy
df_subset["Price"] = df_subset["Price"] * 1.1 # ⚠️ WARNING!

# ✅ Kode yang AMAN:
df_subset = df[df["Quantity"] > 0].copy() # Pasti copy!
df_subset["Price"] = df_subset["Price"] * 1.1 # ✅ Aman!
```

3.4 Apa yang Terjadi Jika TIDAK Menggunakan `df.copy()` ?

Tanpa `df.copy()`, perubahan di dalam fungsi transform **akan mengubah DataFrame asli**:

```
# ❌ TANPA df.copy() – BERBAHAYA
def fix_data_type_BAD(df):
    # Langsung modifikasi df tanpa copy
    df["InvoiceDate"] = pd.to_datetime(df["InvoiceDate"], errors="coerce")
    return df

df_raw = pd.DataFrame({"InvoiceDate": ["2024-01-01", "invalid"]})
df_clean = fix_data_type_BAD(df_raw)

print(df_raw)
# 😱 df_raw JUGA BERUBAH! InvoiceDate sudah jadi datetime!
# Data mentah asli kita hilang selamanya!
```

```
# ✅ DENGAN df.copy() – AMAN (seperti di kode kita)
def fix_data_type(df: pd.DataFrame) -> pd.DataFrame:
    df_new = df.copy() # ✅ Membuat salinan independen
    df_new["InvoiceDate"] = pd.to_datetime(df_new["InvoiceDate"], errors="coerce")
    return df_new

df_raw = pd.DataFrame({"InvoiceDate": ["2024-01-01", "invalid"]})
df_clean = fix_data_type(df_raw)

print(df_raw)
# ✅ df_raw TETAP UTUH! Data asli terjaga!
```

3.5 Dimana `df.copy()` Digunakan di Kode Kita?

Setiap fungsi di `transform.py` **selalu** memulai dengan `df.copy()` :

```
# fix_data_type()
df_new = df.copy() # Baris 12

# standardization_text()
df_new = df.copy() # Baris 29

# handling_missing_values()
df_clean = df.copy() # Baris 57

# handling_duplicates()
df_clean = df.copy() # Baris 87

# transform_all()
df_transform = df.copy() # Baris 123
```


💡 **Best Practice:** Selalu gunakan `df.copy()` di awal setiap fungsi transformasi. Ini memastikan **immutability** — fungsi tidak pernah mengubah input aslinya.

4. Fungsi `standardization_text()` — Penjelasan Baris Per Baris

4.1 Apa Itu Standardisasi Teks dan Mengapa Penting?

Standardisasi teks adalah proses menyeragamkan format teks agar konsisten. Masalah umum yang ditangani:

Masalah	Contoh	Dampak	----- ----- -----	Leading whitespace	" John"	Dianggap berbeda dengan
"John"		Trailing whitespace	"John "		Dianggap berbeda dengan	"John"
		Whitespace di kedua sisi	" John "			
Duplikat tidak terdeteksi		Tab characters	"\tJohn"		Muncul sebagai karakter tak terlihat	

4.2 Kode Lengkap dengan Penjelasan

```
def standardization_text(df: pd.DataFrame) -> pd.DataFrame:
    logger.info("PHASE : STANDARDIZATION TEXT")
    df_new = df.copy()                                # 🗑️ Buat salinan agar tidak ubah aslinya
    total_whitespace = 0                               # 📊 Akumulator untuk total whitespace

    logger.info("Process Clean Whitespace")
    for col in df_new.columns:                          # 🔄 Iterasi setiap kolom
        if df_new[col].dtype == "object":                # 🔍 Cek apakah kolom bertipe teks
            whitespace = df_new[col].notna() & (          # 📋 Boolean mask
                df_new[col] != df_new[col].str.strip()
            )
            count = whitespace.sum()                      # 📈 Hitung jumlah True (ada whitespace)
            total_whitespace += count                     # ➕ Tambahkan ke total

            if count > 0:
                logger.info(f"Total Clear Count : {count} From Columns : {col}")
            else:
                logger.info(f"Not Have Whitespace In Columns : {col}")
            df_new[col] = df_new[col].str.strip()          # ✂️ Hapus whitespace!

    logger.info(f"✅ Whitespace cleaning complete: {total_whitespace} values cleaned")
    return df_new
```

4.3 Penjelasan Detail Setiap Baris Penting

🔴 `df_new[col].dtype == "object"` — Mengapa Hanya Kolom Tipe Object?

Di Pandas, tipe data `"object"` biasanya merepresentasikan **kolom teks (string)**. Kita hanya perlu membersihkan whitespace pada teks, bukan pada angka.

```
# Contoh dtype dari setiap kolom:
# InvoiceNo      → object  ✓ Perlu strip (teks)
# StockCode     → object  ✓ Perlu strip (teks)
# Description    → object  ✓ Perlu strip (teks)
# Quantity      → int64   ✗ Tidak perlu strip (angka)
# InvoiceDate    → object  ✓ Perlu strip (teks – belum dikonversi)
# UnitPrice     → float64 ✗ Tidak perlu strip (angka)
# CustomerID    → float64 ✗ Tidak perlu strip (angka)
# Country       → object  ✓ Perlu strip (teks)
```

Kenapa tidak pakai `isinstance(value, str)` ? Karena kita bekerja dengan **kolom** (Series), bukan nilai individual. `dtype == "object"` adalah cara Pandas mengecek tipe kolom secara keseluruhan.

🔴 Operator `&` (Bitwise AND) di Pandas vs `and`

Di baris ini:

```
whitespace = df_new[col].notna() & (
    df_new[col] != df_new[col].str.strip()
)
```

Kita menggunakan `&` (bitwise AND), **BUKAN** `and`. Ini adalah hal yang sering membingungkan pemula.

| Operator | Digunakan Untuk | Konteks | |-----|-----|-----| | `and` | Operasi logika pada **satu nilai** boolean | `if x > 0 and y > 0:` || `&` | Operasi element-wise pada **Series/Array** boolean | `series_a & series_b` |

```
# ✗ SALAH – akan error!
result = df["col"].notna() and (df["col"] != df["col"].str.strip())
# ValueError: The truth value of a Series is ambiguous

# ✓ BENAR – operasi element-wise pada Series
result = df["col"].notna() & (df["col"] != df["col"].str.strip())
```

⚠️ **Penting:** Saat menggunakan `&` di Pandas, selalu gunakan **tanda kurung** `()` untuk mengelompokkan kondisi karena operator `&` memiliki **precedence lebih tinggi** daripada `==` atau `!=`.

🚩 `df_new[col].notna()` — Mengapa Perlu Mengecek Null Sebelum Strip?

Fungsi `notna()` mengembalikan `True` untuk setiap baris yang **TIDAK** null. Ini penting karena:

```
# Jika ada NaN di kolom:
value = NaN
value.strip() # 🚩 ERROR! Tidak bisa strip NaN!

# notna() memastikan kita HANYA membandingkan baris yang memiliki nilai
# NaN & (apapun) = False – NaN akan dilewati secara aman
```

Contoh cara kerja `notna()` :

```
data = pd.Series([" John ", None, " Budi ", NaN])
data.notna()
# 0      True
# 1     False ← None diabaikan
# 2      True
# 3     False ← NaN diabaikan
```

🚩 `df_new[col].str.strip()` — Apa Yang Dilakukan Strip?

Method `str.strip()` menghapus **whitespace characters** dari **kedua sisi** string (awal dan akhir):

Karakter Yang Dihapus	Representasi	Keterangan
Spasi	" "	
Karakter spasi biasa	"\t"	Tab horizontal
Tab	"\n"	Newline Baris baru
Carriage return	"\r"	Kembali ke awal baris
Form feed	"\f"	Halaman baru
Vertical tab	"\v"	Tab vertikal

```
" Hello World ".strip()      # → "Hello World"
"\t Hello \n".strip()        # → "Hello"
" Hello World ".strip()      # → "Hello World" (spasi di TENGAH tetap ada!)
```

💡 **Perhatikan:** `strip()` hanya menghapus whitespace di **awal** dan **akhir**. Spasi di **tengah** string **tidak** dihapus.

🚩 `.sum()` Pada Boolean Series — `True=1` dan `False=0`

Di Python dan Pandas, nilai boolean bisa dijumlahkan karena:

- `True` dianggap sebagai `1`
- `False` dianggap sebagai `0`

```
whitespace = pd.Series([True, False, True, True, False])
whitespace.sum() # → 3 (karena ada 3 True)
```

Jadi `count = whitespace.sum()` menghitung **berapa banyak baris yang memiliki whitespace** di kolom tersebut.

🚀 Akumulasi `total_whitespace`

Variabel `total_whitespace` diinisialisasi dengan `0` dan ditambahkan setiap kali ditemukan whitespace di sebuah kolom:

```
total_whitespace = 0

# Iterasi 1 (kolom "InvoiceNo"): count = 5 → total_whitespace = 5
# Iterasi 2 (kolom "StockCode"): count = 0 → total_whitespace = 5
# Iterasi 3 (kolom "Description"): count = 12 → total_whitespace = 17
# ...
# Akhir: total_whitespace = total semua whitespace yang ditemukan
```

4.4 Contoh Data Sebelum dan Sesudah Standardisasi

SEBELUM standardisasi:

InvoiceNo	Description	Country
" 536365"	"WHITE HANGING HEART "	"United Kingdom "
"536366 "	" WHITE METAL LANTERN"	" France"
"536367"	"CREAM CUPID HEARTS"	"Germany"

SESUDAH standardisasi:

InvoiceNo	Description	Country
"536365"	"WHITE HANGING HEART"	"United Kingdom"
"536366"	"WHITE METAL LANTERN"	"France"
"536367"	"CREAM CUPID HEARTS"	"Germany"

✅ Semua leading dan trailing whitespace telah dihapus!

5. 🛠️ Fungsi `fix_data_type()` — Penjelasan Baris Per Baris

5.1 Mengapa Fixing Data Type Penting dalam ETL?

Ketika data diekstrak dari file CSV, **semua kolom bisa saja dibaca sebagai teks (string/object)**. Ini menjadi masalah karena:

| Masalah | Contoh | Dampak | |-----|-----|-----| | Tanggal sebagai string | `"12/1/2010 08:26"` → `object` | Tidak bisa filter berdasarkan tanggal, tidak bisa hitung selisih hari | | ID sebagai float | `CustomerID = 17850.0` | Terlihat aneh, sulit untuk join/merge | | Angka sebagai string | `"123"` → `object` | Tidak bisa melakukan operasi matematika |

5.2 Kode Lengkap dengan Penjelasan

```
def fix_data_type(df: pd.DataFrame) -> pd.DataFrame:
    logger.info("PHASE : FIXING DATA TYPE")
    try:
        logger.info("Process Load Dataframe Files")
        df_new = df.copy()                # 🗑️ Salinan aman
        logger.info("Load Successfully")
    except TypeError as e:
        logger.error(f"DF failed to load : {e}")
        raise                            # 🚨 Lempar error ke atas

    logger.info("Process Fixing Data Type")
    for col in df_new.columns:           # 🔄 Iterasi semua kolom
        if col == "InvoiceDate":         # 📅 Konversi ke datetime
            df_new[col] = pd.to_datetime(df_new[col], errors="coerce")
        elif col == "CustomerID":       # 📊 Konversi ke Int64
            df_new[col] = pd.to_numeric(df_new[col], errors="coerce").astype("Int64")
    logger.info("✅ Fixing Data Type Successfully")
    return df_new
```

5.3 `pd.to_datetime()` — Konversi ke Datetime

Apa Itu Datetime?

Datetime adalah tipe data khusus di Python/Pandas yang merepresentasikan **tanggal dan waktu**. Dengan tipe datetime, kita bisa:

- 📅 Menyortir data berdasarkan tanggal
- 📊 Mengelompokkan data per bulan/tahun
- ⌚ Menghitung selisih waktu (misalnya: berapa hari antara dua transaksi)

-  Filter data berdasarkan rentang tanggal

Bagaimana `pd.to_datetime()` Bekerja?

```
pd.to_datetime(df_new["InvoiceDate"], errors="coerce")
```

| Parameter | Nilai | Penjelasan | |-----|-----|-----| | Argumen pertama | `df_new["InvoiceDate"]` | Series yang akan dikonversi | | `errors` | `"coerce"` | Jika konversi gagal, isi dengan `NaT` (Not a Time) |

Opsi untuk parameter `errors` :

| Nilai `errors` | Perilaku | Kapan Digunakan | |-----|-----|-----| | `"raise"` (default) | **Langsung error** jika ada data yang tidak bisa dikonversi | Saat kamu yakin semua data valid | | `"coerce"` | Data yang gagal diubah menjadi `NaT` (null untuk datetime) |  **Paling aman untuk ETL** — data kotor tidak crash pipeline | | `"ignore"` | **Tidak melakukan apa-apa** jika gagal, return input asli | Jarang digunakan |

Contoh Data Sebelum dan Sesudah Konversi Datetime

SEBELUM (tipe `object` / string):

| InvoiceDate (object) | Keterangan | |-----|-----| | `"12/1/2010 08:26"` | Format string biasa | | `"12/1/2010 08:28"` | Format string biasa | | `"not-a-date"` |  Data tidak valid |

SESUDAH (tipe `datetime64[ns]`):

| InvoiceDate (datetime64) | Keterangan | |-----|-----| | `2010-12-01 08:26:00` |  Berhasil dikonversi | | `2010-12-01 08:28:00` |  Berhasil dikonversi | | `NaT` |  Not a Time — akan ditangani di `handling_missing_values()` |

5.4 `pd.to_numeric().astype("Int64")` — Konversi ke Nullable Integer

Baris Kode yang Dimaksud

```
df_new[col] = pd.to_numeric(df_new[col], errors="coerce").astype("Int64")
```

Ada **dua operasi** yang dijalankan secara berurutan (chaining):

1. `pd.to_numeric(..., errors="coerce")` — konversi ke angka, gagal jadi `NaN`
2. `.astype("Int64")` — ubah tipe menjadi Nullable Integer

Perbedaan `int64` (lowercase) vs `Int64` (uppercase/Nullable Integer)

Ini adalah salah satu **detail paling penting** yang sering membingungkan pemula:

Fitur	<code>int64</code> (lowercase)	<code>Int64</code> (uppercase)	Tipe
NumPy integer	<code>np.int64</code>	<code>np.int64</code>	
Pandas Nullable Integer	TIDAK	BISA	
Bisa menampung NaN?	✗ TIDAK	✓ BISA	
Notasi	<code>np.int64</code>	<code>np.int64</code>	
Jika ada NaN	✗ Error atau auto-convert ke <code>float64</code>	✓ Tetap integer dengan <code><NA></code>	

Mengapa ini penting? Bayangkan kolom `CustomerID`:

```
# Data asli mungkin mengandung missing values:
customer_ids = pd.Series([17850, None, 13047, None, 12583])

# ✗ Dengan int64:
customer_ids.astype("int64")
# IntCastingNaError: Cannot convert non-finite values (NA or inf) to integer
# ATAU otomatis jadi float64: [17850.0, NaN, 13047.0, NaN, 12583.0]
# CustomerID jadi desimal? 🤖 Aneh!

# ✓ Dengan Int64 (Nullable):
customer_ids.astype("Int64")
# [17850, <NA>, 13047, <NA>, 12583]
# CustomerID tetap integer, NaN ditampilkan sebagai <NA> ✓
```

Contoh Data Sebelum dan Sesudah Konversi

SEBELUM (tipe `float64`):

CustomerID (float64)	Keterangan
17850.0	Float — ada desimal yang tidak perlu
NaN	Missing value
13047.0	Float — ada desimal yang tidak perlu

SESUDAH (tipe `Int64`):

CustomerID (Int64)	Keterangan
17850	✓ Integer bersih tanpa desimal
<NA>	✓ Missing value tetap terjaga sebagai <NA>
13047	✓ Integer bersih tanpa desimal

5.5 Iterasi `for col in df_new.columns`

```
for col in df_new.columns:
    if col == "InvoiceDate":
        ...
    elif col == "CustomerID":
        ...
```

Kode ini mengiterasi **semua nama kolom** di DataFrame. Untuk setiap kolom, kode mengecek apakah nama kolomnya adalah `"InvoiceDate"` atau `"CustomerID"`, lalu melakukan konversi yang sesuai.

Mengapa menggunakan loop? Meskipun saat ini hanya ada 2 kolom yang dikonversi, pendekatan loop membuat kode **mudah diperluas**. Jika di masa depan ada kolom baru yang perlu dikonversi, kita tinggal menambahkan `elif` baru.

5.6 Error Handling dengan `TypeError`

```
try:
    df_new = df.copy()
except TypeError as e:
    logger.error(f"DF failed to load : {e}")
    raise
```

| Komponen | Penjelasan | |-----|-----| | `try:` | Coba jalankan kode di dalamnya | | `except TypeError as e:` | Jika terjadi `TypeError`, tangkap error-nya | | `logger.error(...)` | Catat error ke log untuk debugging | | `raise` |

Lempar ulang error ke atas — fungsi pemanggil harus menangani |

`TypeError` bisa terjadi jika argumen `df` bukan DataFrame yang valid (misalnya `None`, `int`, atau objek lain yang tidak memiliki method `.copy()`).

💡 **Keyword `raise` tanpa argumen:** Ketika kita menulis `raise` saja (tanpa exception baru), Python akan **melempar ulang exception yang sama** yang sedang ditangkap. Ini berguna karena kita sudah mencatat error ke log, tapi tetap ingin membiarkan error naik ke pemanggil.

6. 🐼 Fungsi `handling_missing_values()` — Penjelasan Baris Per Baris

6.1 Apa Itu Missing Values?

Missing values (nilai yang hilang) adalah sel dalam dataset yang **tidak memiliki data**. Di Pandas, missing values bisa muncul dalam beberapa bentuk:

| Representasi | Tipe Data | Keterangan | |-----|-----|-----| | `NaN` | Float | **Not a Number** — paling umum di Pandas | | `None` | `NoneType` | Objek Python untuk "tidak ada nilai" | | `NaT` | datetime | **Not a Time** — missing value untuk datetime | | `<NA>` | `pd.NA` | Missing value untuk Nullable types (seperti `Int64`) | | `""` (string kosong) | str | ⚠️ **BUKAN** missing value menurut Pandas! |

⚠️ **Penting:** String kosong `""` **TIDAK dianggap** sebagai missing value oleh `isnull()`. Ini adalah kesalahan umum pemula.

6.2 Mengapa Missing Values Berbahaya?

| Dampak | Contoh | |-----|-----| | 📊 **Statistik bias** | Rata-rata harga terhitung salah karena **NaN** diabaikan | | 🔗
Join/Merge gagal | **NaN** tidak bisa dicocokkan dengan key apapun | | 📉 **Visualisasi rusak** | Chart menunjukkan gap/lubang aneh | | 💻 **Machine Learning crash** | Banyak algoritma ML tidak bisa memproses **NaN** |

6.3 Kode Lengkap dengan Penjelasan

```
def handling_missing_values(df: pd.DataFrame) -> pd.DataFrame:
    df_clean = df.copy() # 🗑️ Salinan aman
    logger.info("PHASE : HANDLING MISSING VALUES")

    total_missing_values = df_clean.isnull().sum() # 📊 Hitung missing per kolom
    cols_with_missing_values = [] # 📋 List kolom yang punya missing

    for col, total in total_missing_values.items(): # 🔁 Iterasi hasil
        if total > 0:
            logger.info(f"Column {col} have {total} missing values")
            cols_with_missing_values.append(col) # ➕ Tambah ke list

    rows_before = len(df_clean) # 📏 Catat jumlah baris sebelum

    logger.info("Process Remove Null Values")
    df_clean = df_clean.dropna( # 🗑️ Hapus baris dengan null
        subset=cols_with_missing_values
    )

    rows_after = len(df_clean) # 📏 Catat jumlah baris sesudah
    rows_dropped = rows_before - rows_after # 📉 Hitung baris yang terhapus

    logger.info(f"Drop Missing Values : {rows_dropped} rows")
    logger.info(f"Remaining Rows : {rows_after}")
    logger.info(f"✅ Handling Missing Values Complete")
    return df_clean
```

6.4 `df_clean.isnull().sum()` — Cara Menghitung Missing Values Per Kolom

Mari kita uraikan operasi ini langkah per langkah:

Langkah 1: `df_clean.isnull()` — Menghasilkan DataFrame boolean

```
# Contoh DataFrame:
#   InvoiceNo  CustomerID  Description
# 0  536365    17850.0    WHITE HANGING
# 1  536366     NaN      None
# 2  536367    13047.0    CREAM CUPID

df_clean.isnull()
#   InvoiceNo  CustomerID  Description
# 0  False      False      False
# 1  False      True      True      ← Ada missing values!
# 2  False      False      False
```

Langkah 2: `.sum()` — Jumlahkan `True` (=1) per kolom

```
df_clean.isnull().sum()
# InvoiceNo      0
# CustomerID     1  ← Ada 1 missing value
# Description     1  ← Ada 1 missing value
# dtype: int64
```

6.5 Strategi Handling Missing Values

Ada dua strategi utama untuk menangani missing values. Kode kita menggunakan strategi **Drop**:

Strategi 1: Drop (Menghapus) ← Digunakan di kode kita 

```
df_clean = df_clean.dropna(subset=cols_with_missing_values)
```

Kapan cocok digunakan:

- ✓ Dataset cukup besar (menghapus beberapa baris tidak signifikan)
- ✓ Missing values ada di kolom penting yang tidak bisa diisi sembarangan (misalnya `CustomerID`)
- ✓ Persentase missing values relatif kecil (misalnya < 5%)
- ✓ Mengisi dengan nilai palsu lebih berbahaya daripada menghapus

Kapan TIDAK cocok:

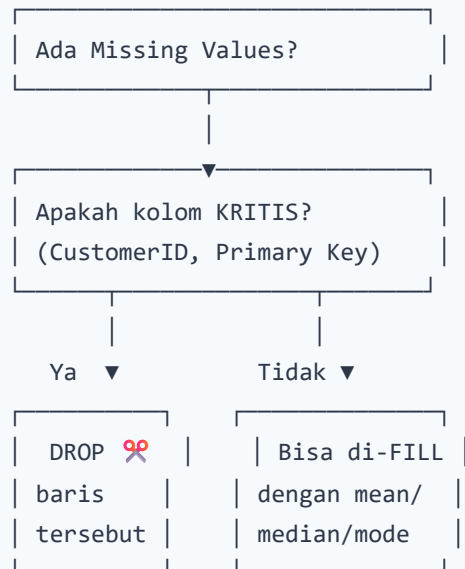
- ✗ Dataset kecil — setiap baris sangat berharga
- ✗ Missing values sangat banyak — bisa kehilangan mayoritas data
- ✗ Missing values memiliki pola (not random) — menghapus bisa menyebabkan bias

Strategi 2: Fill/Imputation (Mengisi) — Alternatif

Meskipun **tidak digunakan** di kode kita, penting untuk mengetahui alternatif ini:

Metode Fill	Kode	Keterangan	Mean (rata-rata)
<code>df["col"].fillna(df["col"].mean())</code>		Cocok untuk distribusi normal	Median (nilai tengah)
<code>df["col"].fillna(df["col"].median())</code>		Cocok untuk data dengan outlier	Mode (nilai terbanyak)
<code>df["col"].fillna(df["col"].mode()[0])</code>		Cocok untuk data kategorik	Forward Fill
<code>df["col"].fillna(method="ffill")</code>		Isi dengan nilai baris sebelumnya (untuk time series)	Backward Fill
<code>df["col"].fillna(method="bfill")</code>		Isi dengan nilai baris sesudahnya	Nilai Konstan <code>df["col"].fillna(0)</code> Isi dengan nilai tetap

Kapan Memilih Drop vs Fill?



6.6 `df_clean.dropna(subset=cols_with_missing_values)` — Parameter `subset`

Parameter `subset` menentukan **kolom mana saja** yang dijadikan acuan untuk menghapus baris:

```

# Tanpa subset – hapus baris yang PUNYA NaN di kolom MANAPUN:
df.dropna()

# Dengan subset – hapus baris yang punya NaN HANYA di kolom tertentu:
df.dropna(subset=["CustomerID", "Description"])

# Baris hanya dihapus jika CustomerID ATAU Description adalah NaN
# Kolom lain boleh punya NaN dan baris tetap dipertahankan
  
```

Di kode kita, `cols_with_missing_values` berisi **hanya kolom yang memang memiliki missing values**, sehingga kita tidak menghapus baris secara berlebihan.

6.7 Mengapa Menyimpan `rows_before` dan `rows_after` ?

```
rows_before = len(df_clean)          # Misalnya: 541909
df_clean = df_clean.dropna(...)
rows_after = len(df_clean)           # Misalnya: 406829
rows_dropped = rows_before - rows_after # 135080 baris dihapus
```

Ini penting untuk **transparansi dan audit**:

1. **Audit Trail**: Kita bisa memverifikasi berapa banyak data yang hilang
2. **Quality Check**: Jika `rows_dropped` terlalu besar, mungkin ada masalah di data atau di logika
3. **Monitoring**: Tim data bisa memantau tren missing values dari waktu ke waktu
4. **Debugging**: Jika hasil akhir tidak sesuai ekspektasi, kita bisa melacak di langkah mana data berkurang drastis

6.8 Contoh Data Sebelum dan Sesudah Handling

SEBELUM handling missing values (setelah `fix_data_type`):

InvoiceNo	Description	CustomerID	Quantity						
536365	WHITE HANGING HEART	17850	6	536366	WHITE METAL LANTERN	<NA>	2	536367	NaN
13047	8	536368	CREAM CUPID HEARTS	12583	3				

SESUDAH handling (baris dengan missing values dihapus):

InvoiceNo	Description	CustomerID	Quantity						
536365	WHITE HANGING HEART	17850	6	536368	CREAM CUPID HEARTS	12583	3		

✅ Baris ke-2 dihapus karena `CustomerID` = `<NA>` ✅ Baris ke-3 dihapus karena `Description` = `NaN`

7. Fungsi `handling_duplicates()` — Penjelasan Baris Per Baris

7.1 Apa Itu Data Duplikat dan Mengapa Berbahaya?

Data duplikat adalah baris-baris yang memiliki **nilai identik di SEMUA kolom**. Duplikat biasanya muncul karena:

Penyebab	Contoh
Import berulang	Data CSV di-import dua kali
Retry mechanism	Request API diulang saat timeout
Bug di sumber data	Sistem sumber menghasilkan duplikat
Input manual	User submit form dua kali

Dampak duplikat pada analisis:

Tanpa penghapusan duplikat:

Total Penjualan = 1000 + 1000 + 500 = 2500 ← 1000 terhitung DUA KALI! 🤖

Setelah penghapusan duplikat:

Total Penjualan = 1000 + 500 = 1500 ← Akurat! ✅

7.2 Kode Lengkap dengan Penjelasan

```
def handling_duplicates(df: pd.DataFrame) -> pd.DataFrame:
    logger.info("PHASE : HANDLING DUPLICATES")

    df_clean = df.copy()                # 🗑️ Salinan aman
    rows_before = len(df_clean)         # 📝 Catat jumlah awal

    total_duplicates = df.duplicated().sum() # 📊 Hitung total duplikat
    logger.info(f"Total Duplicates Values : {total_duplicates}")

    if total_duplicates == 0:           # ⚡ Early return jika 0
        logger.info("✅ Not Have Duplicate Values")
        return df_clean

    logger.info("Process Drop Duplicate Values")
    df_clean = df_clean.drop_duplicates() # 🗑️ Hapus duplikat

    rows_after = len(df_clean)          # 📝 Catat jumlah sesudah
    rows_dropped = rows_before - rows_after # 📉 Hitung yang terhapus

    logger.info(f"Drop Duplicate Values : {rows_dropped} rows")
    logger.info(f"Remaining Rows : {rows_after}")
    logger.info("✅ Handling Duplicates Complete")
    return df_clean
```

7.3 `df.duplicated()` — Cara Kerja Deteksi Duplikat

Method `duplicated()` mengembalikan **Series boolean** yang menandai apakah setiap baris adalah duplikat:

```
df = pd.DataFrame({
    "Nama": ["Andi", "Budi", "Andi", "Charlie", "Budi"],
    "Umur": [25, 30, 25, 35, 30]
})

df.duplicated()
# 0    False  < Andi/25 - kemunculan PERTAMA (bukan duplikat)
# 1    False  < Budi/30 - kemunculan PERTAMA
# 2     True  < Andi/25 - DUPLIKAT dari baris 0! ⚠️
# 3    False  < Charlie/35 - unik
# 4     True  < Budi/30 - DUPLIKAT dari baris 1! ⚠️
```

Secara default, `duplicated()` menandai **kemunculan kedua dan seterusnya** sebagai duplikat. Kemunculan pertama dianggap asli (original).

Parameter penting:

| Parameter | Nilai Default | Penjelasan | |-----|-----|-----| | `keep` | `"first"` | Tandai semua duplikat KECUALI kemunculan **pertama** | | `keep="last"` | — | Tandai semua duplikat KECUALI kemunculan **terakhir** | | `keep=False` | — | Tandai **SEMUA** baris yang memiliki duplikat (termasuk yang pertama) | | `subset` | Semua kolom | Hanya periksa kolom tertentu untuk menentukan duplikat |

7.4 `.sum()` Untuk Menghitung Total Duplikat

```
total_duplicates = df.duplicated().sum()
```

Sama seperti di `standardization_text()`, `.sum()` menghitung jumlah `True` dalam Series boolean:

```
pd.Series([False, False, True, False, True]).sum()
# → 2 (ada 2 duplikat)
```

7.5 Early Return Pattern

```
if total_duplicates == 0:
    logger.info("✅ Not Have Duplicate Values")
    return df_clean
```

Early return (atau **guard clause**) adalah pattern di mana kita **keluar dari fungsi lebih awal** jika kondisi tertentu terpenuhi. Mengapa ini efisien?

1. **Menghindari operasi yang tidak perlu:** Jika tidak ada duplikat, kenapa menjalankan `drop_duplicates()` ?
2. **Kode lebih mudah dibaca:** Kasus spesial (tidak ada duplikat) ditangani di awal, logika utama tetap bersih
3. **Performance:** `drop_duplicates()` bisa mahal untuk dataset besar. Jika tidak ada duplikat, kita skip

```
# ❌ TANPA early return – kurang efisien
def handling_duplicates(df):
    df_clean = df.copy()
    total = df.duplicated().sum()
    if total > 0:
        df_clean = df_clean.drop_duplicates() # Nested di dalam if
        # ... logging ...
    return df_clean

# ✅ DENGAN early return – lebih bersih dan efisien
def handling_duplicates(df):
    df_clean = df.copy()
    total = df.duplicated().sum()
    if total == 0:
        return df_clean # ⚡ Keluar cepat!
    df_clean = df_clean.drop_duplicates() # Tidak perlu nested
    # ... logging ...
    return df_clean
```

7.6 `df_clean.drop_duplicates()` — Parameter Default

```
df_clean = df_clean.drop_duplicates()
```

Tanpa argumen, `drop_duplicates()` menggunakan parameter default:

Parameter	Default	Penjelasan
<code>subset</code>	Semua kolom	Semua kolom digunakan untuk menentukan duplikat
<code>keep</code>	"first"	✔️ Pertahankan kemunculan pertama , hapus sisanya
<code>inplace</code>	False	Mengembalikan DataFrame baru (tidak mengubah asli)
<code>ignore_index</code>	False	Index asli dipertahankan

Karena `keep="first"` adalah default, kode kita **mempertahankan baris pertama** dari setiap grup duplikat dan menghapus yang lainnya.

7.7 Contoh Data Sebelum dan Sesudah Handling

SEBELUM handling duplicates:

Index	InvoiceNo	Description	CustomerID	Quantity
536365	WHITE HANGING HEART	17850	6	1
536366	WHITE METAL LANTERN	13047	2	2
536365	WHITE			

HANGING HEART | 17850 | 6 | | 3 | 536368 | CREAM CUPID HEARTS | 12583 | 3 | | 4 | 536366 | WHITE METAL LANTERN | 13047 | 2 |

SESUDAH handling (duplikat dihapus, keep='first'):

| Index | InvoiceNo | Description | CustomerID | Quantity | |-----|-----|-----|-----|-----| | 0 |
536365 | WHITE HANGING HEART | 17850 | 6 | | 1 | 536366 | WHITE METAL LANTERN | 13047 | 2 | | 3 | 536368 | CREAM
CUPID HEARTS | 12583 | 3 |

✅ Baris index 2 dihapus (duplikat dari baris 0) ✅ Baris index 4 dihapus (duplikat dari baris 1) ✅

Perhatikan index tetap dipertahankan (0, 1, 3) — bukan di-reset

8. 🎯 Fungsi `transform_all()` — Orchestrator Pipeline

8.1 Apa Itu Orchestrator/Pipeline Function?

Orchestrator function (fungsi orkestrator) adalah fungsi yang **mengatur dan mengkoordinasikan** pemanggilan fungsi-fungsi lain dalam urutan yang benar. Ia tidak melakukan transformasi data secara langsung, melainkan **mendelegasikan** pekerjaan ke fungsi-fungsi spesifik.

Bayangkan analogi **konduktor orkestra**: konduktor tidak memainkan alat musik apapun, tapi ia memastikan setiap musisi bermain di waktu yang tepat dan dalam urutan yang benar.

8.2 Kode Lengkap dengan Penjelasan

```
def transform_all(df: pd.DataFrame) -> pd.DataFrame:
    """
    Transform all data from DataFrame.

    Args:
        df (pd.DataFrame): DataFrame from the CSV file.

    Returns:
        pd.DataFrame: DataFrame from the CSV file.
    """
    print("=" * 60)                                # 🚩 Header visual
    print("PHASE 2: TRANSFORMATION")
    print("=" * 60)
    df_transform = df.copy()                        # 🗝 Salinan aman

    # Pipeline Transform Phase
    df_transform = standardization_text(df_transform) # 1 Bersihkan whitespace
    df_transform = fix_data_type(df_transform)        # 2 Perbaiki tipe data
    df_transform = handling_missing_values(df_transform) # 3 Tangani missing values
    df_transform = handling_duplicates(df_transform)  # 4 Tangani duplikat

    print("=" * 60)                                # 🚩 Footer visual
    print("TRANSFORMATION IS SUCCESSFULLY")
    print("=" * 60)
    return df_transform
```

8.3 Pattern: Input → Process → Output (Chaining)

Perhatikan bagaimana **output dari satu fungsi menjadi input untuk fungsi berikutnya**:

```

df_transform (raw)
  |
  ▼
standardization_text(df_transform) → df_transform (whitespace bersih)
  |
  ▼
fix_data_type(df_transform) → df_transform (tipe data benar)
  |
  ▼
handling_missing_values(df_transform) → df_transform (tanpa NaN)
  |
  ▼
handling_duplicates(df_transform) → df_transform (tanpa duplikat)
  |
  ▼
return df_transform (BERSIH ✅)

```

Ini disebut **transformation chaining** — setiap langkah menyempurnakan DataFrame secara bertahap. Variabel `df_transform` di-reassign setiap kali, sehingga selalu menunjuk ke versi terbaru DataFrame.

```

# Ini BUKAN chaining method (seperti df.strip().lower())
# Ini adalah SEQUENTIAL reassignment:
df_transform = fungsi_1(df_transform) # Hasilnya ditampung ulang
df_transform = fungsi_2(df_transform) # Hasilnya ditampung ulang lagi
df_transform = fungsi_3(df_transform) # Dan seterusnya...

```

8.4 Mengapa Menggunakan `print()` untuk Header, Bukan `logger` ?

Perhatikan bahwa `transform_all()` menggunakan `print()`, sementara fungsi-fungsi lain menggunakan `logger.info()`:

Fungsi	Output	Alasan	----- ----- -----	<code>print()</code>	Langsung ke console (stdout)	Untuk user-facing output — visual feedback saat menjalankan script
				<code>logger.info()</code>	Ke log handler (file/console tergantung konfigurasi)	Untuk developer/audit — catatan teknis yang bisa dikonfigurasi level-nya

`print()` pada header/footer pipeline memberikan **feedback visual** yang jelas kepada user bahwa fase Transform telah dimulai dan selesai:

```

=====
PHASE 2: TRANSFORMATION
=====
... (proses berlangsung) ...
=====
TRANSFORMATION IS SUCCESSFULLY
=====

```

9. 🚩 Blok `if __name__ == "__main__"`

9.1 Kode Lengkap

```

if __name__ == "__main__":
    project_root = Path(__file__).resolve().parent.parent
    if str(project_root) not in sys.path:
        sys.path.insert(0, str(project_root))

    from src.extract import extract_get_data
    df = extract_get_data(project_root)

    df = transform_all(df)

```

9.2 Apa Itu `if __name__ == "__main__"` ?

Blok ini adalah **idiom Python standar** yang memastikan kode di dalamnya **hanya berjalan ketika file dijalankan langsung**, bukan ketika di-import sebagai modul.

```

# Skenario 1: Menjalankan langsung
python src/transform.py
# __name__ = "__main__" → Blok DIJALANKAN ✅

# Skenario 2: Di-import dari file lain
from src.transform import transform_all
# __name__ = "src.transform" → Blok TIDAK DIJALANKAN ❌

```

Ini sangat berguna karena `transform.py` bisa digunakan dalam **dua cara**:

1. **Sebagai modul** — di-import oleh pipeline utama (`main.py`)

2. Sebagai script mandiri — dijalankan langsung untuk testing/debugging

9.3 `sys.path` Manipulation — Mengapa Diperlukan?

```
project_root = Path(__file__).resolve().parent.parent
if str(project_root) not in sys.path:
    sys.path.insert(0, str(project_root))
```

Mari kita uraikan baris per baris:

`Path(__file__).resolve().parent.parent`

<code>Path(__file__)</code>	# Path ke file saat ini: <code>src/transform.py</code>
<code>.resolve()</code>	# Path absolut: <code>C:/Users/.../ETL_Pipeline_Python/src/transform.py</code>
<code>.parent</code>	# Naik 1 level: <code>C:/Users/.../ETL_Pipeline_Python/src/</code>
<code>.parent</code>	# Naik 2 level: <code>C:/Users/.../ETL_Pipeline_Python/</code> ← ROOT!

`sys.path.insert(0, str(project_root))`

`sys.path` adalah **list** yang berisi direktori-direktori tempat Python mencari modul saat `import`. Dengan menambahkan `project_root` ke awal `sys.path`, kita memastikan Python bisa menemukan modul `src.extract`.

```
# Tanpa sys.path manipulation:
from src.extract import extract_get_data # ❌ ModuleNotFoundError!
# Python tidak tahu di mana "src" berada

# Dengan sys.path manipulation:
sys.path.insert(0, "C:/Users/.../ETL_Pipeline_Python")
from src.extract import extract_get_data # ✅ Berhasil!
# Python sekarang tahu mencari di root project
```

Guard condition: `if str(project_root) not in sys.path`

Pengecekan ini mencegah **penambahan path yang sudah ada** — menghindari duplikasi di `sys.path`.

9.4 Import dari Modul Lain dan Cara Menjalankan Standalone

```
from src.extract import extract_get_data
df = extract_get_data(project_root)
df = transform_all(df)
```

Urutan eksekusi saat dijalankan standalone:

1. **Import** fungsi `extract_get_data` dari modul `src.extract`
2. **Extract**: Panggil `extract_get_data()` untuk mendapatkan DataFrame mentah
3. **Transform**: Panggil `transform_all()` untuk membersihkan DataFrame

Untuk menjalankan `transform.py` secara standalone dari terminal:

```
# Dari root project:
python src/transform.py

# Atau menggunakan module syntax:
python -m src.transform
```

10. Konsep Python Yang Digunakan

10.1 DataFrame Operations

Berikut adalah semua operasi DataFrame yang digunakan di `transform.py`:

Operasi	Contoh di Kode	Penjelasan	-----	-----	-----		<code>df.copy()</code>	<code>df_new = df.copy()</code>
Membuat deep copy DataFrame	<code>df.dropna()</code>	<code>df_clean.dropna(subset=...)</code>	Menghapus baris dengan NaN		<code>df.drop_duplicates()</code>	<code>df_clean.drop_duplicates()</code>	Menghapus baris duplikat	<code>df.isnull()</code>
<code>df_clean.isnull().sum()</code>	Mengecek missing values	<code>df.duplicated()</code>	<code>df.duplicated().sum()</code>	Mengecek baris duplikat	<code>df.columns</code>	<code>for col in df_new.columns</code>	Mendapatkan list nama kolom	<code>df[col].dtype</code>
<code>df_new[col].dtype == "object"</code>	Mengecek tipe data kolom	<code>df[col].notna()</code>	<code>df_new[col].notna()</code>	Kebalikan dari <code>isnull()</code>	<code>df[col].str.strip()</code>	<code>df_new[col].str.strip()</code>	Strip whitespace dari teks	
<code>len(df)</code>	<code>rows_before = len(df_clean)</code>	Menghitung jumlah baris						

10.2 Boolean Indexing dan Masking

Boolean masking adalah teknik di mana kita menggunakan **Series boolean** untuk memfilter atau menganalisis data:

```
# Membuat boolean mask untuk mendeteksi whitespace
whitespace = df_new[col].notna() & (
    df_new[col] != df_new[col].str.strip()
)

# Hasilnya adalah Series boolean:
# 0    False    <- "John" == "John".strip() -> sama, tidak ada whitespace
# 1     True     <- " Budi " != "Budi" -> berbeda, ada whitespace!
# 2    False    <- NaN -> notna() = False -> selalu False
```

Boolean mask bisa digunakan untuk:

```
# 1. Menghitung jumlah True
count = mask.sum()

# 2. Memfilter DataFrame
df_filtered = df[mask]

# 3. Mengganti nilai tertentu
df.loc[mask, "col"] = new_value
```

10.3 Operator Bitwise & vs Logical and

Operator	Tipe	Untuk	Contoh					
<code>series_a & series_b</code>	Bitwise AND	Series/Array boolean	<code>if x > 0 and y > 0:</code>	<code> </code>	<code>\\</code>	Bitwise OR		
<code>series_a & series_b</code>	Logical AND	Satu nilai boolean	<code>if x > 0 or y > 0:</code>	<code> </code>	<code>~</code>	Bitwise NOT	Series/Array boolean	
<code>series_a & series_b</code>	Logical OR	Satu nilai boolean	<code>if not x:</code>			Logical NOT	Satu nilai boolean	

```
# ❌ SALAH: Menggunakan `and` untuk Series
result = df["A"].notna() and df["B"].notna()
# ValueError: The truth value of a Series is ambiguous.

# ✅ BENAR: Menggunakan `&` untuk Series
result = df["A"].notna() & df["B"].notna()
```

⚠️ **Selalu gunakan tanda kurung** saat menggabungkan kondisi dengan `&` atau `|`:

```
# Wajib pakai kurung karena precedence & lebih tinggi dari ==
result = (df["A"] > 0) & (df["B"] < 10) # ✅
result = df["A"] > 0 & df["B"] < 10    # ❌ Hasil tidak sesuai!
```

10.4 String Methods di Pandas (`.str` Accessor)

Pandas menyediakan **accessor** `.str` untuk operasi string pada Series:

```
# Accessor .str memberikan akses ke semua method string Python
df["col"].str.strip()      # Hapus whitespace di awal dan akhir
df["col"].str.lower()      # Ubah ke huruf kecil
df["col"].str.upper()      # Ubah ke huruf besar
df["col"].str.contains()   # Cek apakah mengandung substring
df["col"].str.replace()    # Ganti substring
df["col"].str.len()        # Hitung panjang string
```

Di kode kita, `.str.strip()` digunakan untuk menghapus whitespace:

```
# Sebelum strip:
# Series: [" John ", "Budi", " Charlie"]

df_new[col].str.strip()

# Sesudah strip:
# Series: ["John", "Budi", "Charlie"]
```

 **Penting:** `.str` accessor hanya bisa digunakan pada kolom bertipe `object` (string). Menggunakannya pada kolom numerik akan menghasilkan `NaN` atau error.

10.5 Nullable Types (`Int64` vs `int64`)

Pandas memperkenalkan **Nullable Integer Type** untuk mengatasi keterbatasan NumPy integer yang tidak bisa menampung `NaN`:

```
# NumPy int64 (lowercase):
pd.array([1, 2, None], dtype="int64")
# ❌ TypeError: int() argument must be a string, not NoneType
# Atau otomatis dikonversi ke float64: [1.0, 2.0, NaN]

# Pandas Int64 (uppercase – Nullable):
pd.array([1, 2, None], dtype="Int64")
# ✅ [1, 2, <NA>]
# Integer tetap integer, missing value sebagai <NA>
```

Daftar Nullable Types di Pandas:

| Tipe Nullable | Tipe NumPy Setara | Ukuran | |-----|-----|-----| | `Int8` | `int8` | 8-bit integer | |
`Int16` | `int16` | 16-bit integer | | `Int32` | `int32` | 32-bit integer | | `Int64` | `int64` | 64-bit integer | | `UInt8` |
`uint8` | 8-bit unsigned integer | | `Float32` | `float32` | 32-bit float | | `Float64` | `float64` | 64-bit float | | `boolean` |
`bool` | Boolean nullable | | `string` | `object` | String nullable |

10.6 `errors="coerce"` Pattern

Pattern `errors="coerce"` digunakan pada fungsi konversi Pandas untuk **menangani data yang tidak bisa dikonversi secara aman**:

```
# Pattern ini digunakan di dua tempat:
pd.to_datetime(series, errors="coerce")    # Gagal → NaT
pd.to_numeric(series, errors="coerce")     # Gagal → NaN
```

Ini adalah **safe conversion pattern** — alih-alih menghentikan seluruh pipeline karena satu data yang buruk, kita mengubahnya menjadi missing value yang bisa ditangani di langkah berikutnya.

```
Data "abc" → pd.to_numeric(errors="coerce") → NaN → dropna() → Dihapus
Data "123" → pd.to_numeric(errors="coerce") → 123 → dropna() → Tetap ✓
```

10.7 Early Return Pattern

Early return (juga dikenal sebagai **guard clause**) adalah pattern di mana fungsi **keluar lebih awal** jika kondisi tertentu terpenuhi, menghindari nested code yang tidak perlu:

```
# ✓ Digunakan di handling_duplicates():
if total_duplicates == 0:
    logger.info("✓ Not Have Duplicate Values")
    return df_clean # ⚡ Keluar cepat!

# Kode di bawah ini hanya dijalankan jika ADA duplikat
df_clean = df_clean.drop_duplicates()
```

Keuntungan early return:

1. **Mengurangi nesting** — kode lebih datar dan mudah dibaca
2. **Jelas**: kasus spesial ditangani di awal
3. **Efisien**: menghindari operasi yang tidak perlu

11. Data Quality Metrics

11.1 Apa Saja Metrik Kualitas Data?

Data Quality Metrics (metrik kualitas data) adalah **angka-angka terukur** yang menunjukkan seberapa bersih dan andal data kita. Kode `transform.py` secara aktif melacak beberapa metrik penting:

Metrik	Dilacak Di	Cara Mengukur	Kode
 Whitespace Count	<code>standardization_text()</code>	Jumlah sel yang mengandung whitespace berlebih	<code>total_whitespace += count</code> 
Missing Values Count	<code>handling_missing_values()</code>	Jumlah NaN/None/NaT per kolom	<code>df_clean.isnull().sum()</code>
 Duplicate Count	<code>handling_duplicates()</code>	Jumlah baris duplikat	<code>df.duplicated().sum()</code>  Rows Dropped (Missing) <code>handling_missing_values()</code> Baris yang dihapus karena missing values <code>rows_before - rows_after</code>
 Rows Dropped (Duplicates)	<code>handling_duplicates()</code>	Baris yang dihapus karena duplikasi	<code>rows_before - rows_after</code>  Remaining Rows Kedua fungsi Total baris tersisa setelah pembersihan <code>rows_after</code>

11.2 Bagaimana Kode Ini Melacak Metrik

Setiap fungsi menggunakan **logging** untuk mencatat metrik yang dihasilkan:

```
# standardization_text() – Melacak whitespace
logger.info(f"Total Clear Count : {count} From Columns : {col}")
logger.info(f"✅ Whitespace cleaning complete: {total_whitespace} values cleaned")

# handling_missing_values() – Melacak missing values
logger.info(f"Column {col} have {total} missing values")
logger.info(f"Drop Missing Values : {rows_dropped} rows")
logger.info(f"Remaining Rows : {rows_after}")

# handling_duplicates() – Melacak duplikat
logger.info(f"Total Duplicates Values : {total_duplicates}")
logger.info(f"Drop Duplicate Values : {rows_dropped} rows")
logger.info(f"Remaining Rows : {rows_after}")
```

11.3 Mengapa Logging Metrik Penting untuk Audit Trail?

Audit trail adalah catatan kronologis yang mendokumentasikan **apa yang terjadi** pada data selama proses transformasi. Ini penting karena:

1. **Reproducibility:** Kita bisa memverifikasi bahwa pipeline menghasilkan hasil yang sama setiap kali dijalankan
2. **Debugging:** Jika ada masalah di output, kita bisa melacak **di langkah mana** data berubah
3. **Compliance:** Banyak industri (keuangan, kesehatan) mewajibkan catatan perubahan data
4. **Monitoring:** Tim data bisa memantau **tren kualitas data** dari waktu ke waktu

Contoh output log yang dihasilkan:

```
INFO - PHASE : STANDARDIZATION TEXT
INFO - Process Clean Whitespace
INFO - Total Clear Count : 1148 From Columns : Description
INFO - Not Have Whitespace In Columns : InvoiceNo
INFO - ✅ Whitespace cleaning complete: 1148 values cleaned
INFO - PHASE : FIXING DATA TYPE
INFO - Process Fixing Data Type
INFO - ✅ Fixing Data Type Successfully
INFO - PHASE : HANDLING MISSING VALUES
INFO - Column Description have 1454 missing values
INFO - Column CustomerID have 135080 missing values
INFO - Drop Missing Values : 135080 rows
INFO - Remaining Rows : 406829
INFO - ✅ Handling Missing Values Complete
INFO - PHASE : HANDLING DUPLICATES
INFO - Total Duplicates Values : 5225
INFO - Drop Duplicate Values : 5225 rows
INFO - Remaining Rows : 401604
INFO - ✅ Handling Duplicates Complete
```

11.4 Visualisasi Alur Data dan Jumlah Baris

Berikut gambaran bagaimana jumlah baris berubah selama pipeline transformasi (contoh dari dataset nyata):



Data Retention Rate: 401,604 / 541,909 = **74.1%** data berhasil dipertahankan setelah pembersihan. Ini menunjukkan bahwa 25.9% data asli tidak memenuhi standar kualitas.

12. ✓ Ringkasan & Poin-Poin Penting

12.1 Checklist Apa yang Sudah Dipelajari

Setelah mempelajari dokumentasi ini, kamu seharusnya sudah memahami:

- [x] **Apa itu fase Transform** dan mengapa ini adalah fase paling kompleks dalam ETL
- [x] **Prinsip GIGO** (Garbage In, Garbage Out) dan pentingnya data bersih
- [x] **Pipeline transformasi** dan mengapa urutan langkah sangat penting
- [x] `df.copy()` — mutable vs immutable, deep copy, dan menghindari side effects
- [x] `standardization_text()` — strip whitespace, boolean masking, operator `&`
- [x] `fix_data_type()` — `pd.to_datetime()`, `pd.to_numeric()`, `errors="coerce"`, `Int64` vs `int64`
- [x] `handling_missing_values()` — deteksi NaN, strategi drop vs fill, parameter `subset`
- [x] `handling_duplicates()` — `duplicated()`, `drop_duplicates()`, early return pattern
- [x] `transform_all()` — orchestrator pattern, chaining transformations
- [x] `if __name__ == "__main__"` — standalone execution, `sys.path` manipulation
- [x] **Konsep Python** — boolean indexing, `.str` accessor, nullable types
- [x] **Data quality metrics** — audit trail, monitoring, dan transparansi

12.2 Tips dan Best Practices untuk Transform Phase

Arsitektur & Design

| # | Best Practice | Diterapkan di Kode? | |---|-----|-----| | 1 | Pisahkan setiap jenis transformasi ke fungsi tersendiri | ✓ Ya — 4 fungsi terpisah | | 2 | Buat fungsi orchestrator untuk mengatur urutan | ✓ Ya — `transform_all()` | | 3 | Selalu gunakan `df.copy()` untuk menghindari side effects | ✓ Ya — di setiap fungsi | | 4 | Tentukan urutan transformasi dengan sengaja dan logis | ✓ Ya — standardize → fix type → handle missing → handle duplicates |

Data Quality

| # | Best Practice | Diterapkan di Kode? | |---|-----|-----| | 5 | Catat jumlah data sebelum dan sesudah transformasi | ✓ Ya — `rows_before`, `rows_after` | | 6 | Log setiap langkah transformasi untuk audit trail | ✓ Ya — logging di setiap fungsi | | 7 | Gunakan `errors="coerce"` untuk safe conversion | ✓ Ya — di `fix_data_type()` | | 8 | Gunakan early return untuk efisiensi | ✓ Ya — di `handling_duplicates()` |

Error Handling

| # | Best Practice | Diterapkan di Kode? | |---|-----|-----| | 9 | Tangkap exception spesifik (bukan generic `Exception`) | ☒ Ya — `TypeError` | | 10 | Log error sebelum re-raise | ☒ Ya — `logger.error(...)` lalu `raise` | | 11 | Gunakan type hints untuk dokumentasi parameter | ☒ Ya — `df: pd.DataFrame -> pd.DataFrame` |

Tips Tambahan untuk Learners

1. **Selalu eksplorasi data terlebih dahulu** sebelum menulis transformasi:

```
df.info()          # Lihat tipe data dan missing values
df.describe()      # Statistik dasar
df.head()          # Lihat sampel data
df.isnull().sum()  # Hitung missing values
df.duplicated().sum() # Hitung duplikat
```

2. **Validasi hasil transformasi** setelah setiap langkah:

```
print(f"Shape: {df.shape}")          # Pastikan jumlah baris/kolom sesuai
print(f"Dtypes:\n{df.dtypes}")       # Pastikan tipe data sudah benar
print(f"Nulls:\n{df.isnull().sum()}") # Pastikan tidak ada missing values
```

3. **Simpan data intermediate** saat debugging:

```
df_after_step1.to_csv("debug_step1.csv") # Simpan setelah langkah 1
df_after_step2.to_csv("debug_step2.csv") # Simpan setelah langkah 2
```

4. **Jangan takut untuk menghapus data** yang tidak valid. Lebih baik memiliki 1000 baris data bersih daripada 10,000 baris data kotor.
5. **Dokumentasikan keputusan transformasi.** Mengapa memilih `drop` daripada `fill`? Mengapa `Int64` bukan `int64`? Catatan ini akan sangat berharga di masa depan.



Catatan Akhir: Dokumentasi ini dibuat sebagai bagian dari perjalanan belajar ETL Pipeline menggunakan Python. Fase Transform adalah fase yang paling banyak membutuhkan pemahaman domain dan keputusan desain. Setiap dataset unik, dan transformasi yang diperlukan akan berbeda-beda. Kode di `transform.py` memberikan fondasi yang kuat untuk menangani masalah kualitas data yang paling umum: whitespace, tipe data salah, missing values, dan duplikat.

 Dokumentasi ini adalah bagian dari seri pembelajaran ETL Pipeline Python. Fase sebelumnya: Extract | Fase selanjutnya: Load